



Building Storage Analytics Pipelines for Cloud Cost Optimization with Airflow



Bao Nguyen

A Best Picture winner was reviewed on Frame.io.

Every cut. Every draft. Every version of every scene — stored.

It all adds to the cloud bill. The bill never says which.



What we'll cover

- 1 Why “storage” isn't one number
- 2 A general framework: lifecycle, point-in-time modeling, the DAG
- 3 Three things that make a pipeline trustworthy
- 4 What changed once we got it right
- 5 Takeaways you can reuse

Not all "storage" numbers agree

The business question: is spend tracking real, meaningful usage — or is there something to optimize?
Say your team uploads 5 videos to a project...

WHAT YOU SEE

Objects uploaded

"5 new videos this week"

Events — not what's still stored

WHAT YOU PAY

Billed cloud storage

The invoice

Replication, backups, DR mixed in

WHAT'S REAL

Assets Under Management

The best proxy for cost-bearing usage, day by day

None of these are wrong to compute. Only one answers the business question.

Assets Under Management, precisely

THE DEFINITION

Uploaded on or before the day

Not hard-deleted before the day

Sum of file sizes = that day's AUM

THE DECISIONS

Soft-deleted still counts (retention)

Point-in-time → clean YoY

Every include/exclude written down

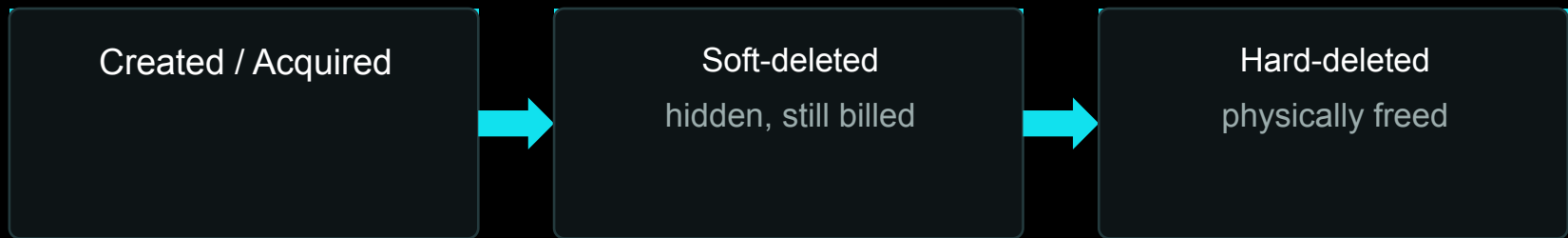
AUM growth $\approx$ meaningful usage growth.

4-5x

How far billed cost can run above AUM — replication, backups, and DR make up the gap.

The ratio is the signal: stable means known overhead. Drifting means inefficiency to hunt down.

Model the full resource lifecycle

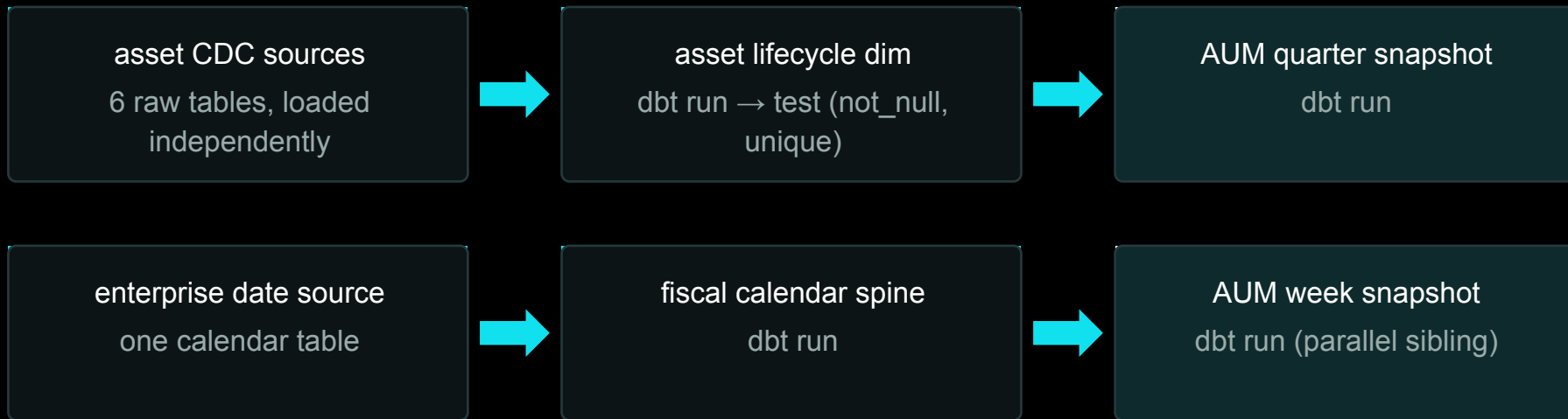


A soft-deleted resource is still stored — and billed — during the retention window.

Storage is a stock, not a flow.

To know what customers are actually storing at any moment, you can't sum events – you take stock, at regular intervals. That's the pipeline we run with Airflow.

What the DAG actually looks like



Cosmos renders our dbt project as Airflow tasks: each model becomes a task, models with tests become run → test pairs.

Three principles that outlast any DAG

1

Right

Definitions in code, tested in the DAG

2

Fast

No wasted fan-out — persist snapshots

3

Useful

Report what answers the question

So... Finance was happy.

On autopilot

What used to be all-day DMs
with Finance now answers
itself

Forecast matched reality

Held up when Finance
checked independently

Retention got cheaper

Invisible-but-billed storage
informed a policy change

The best status update is the one nobody has to ask for. 🌴

Key takeaways

1

AUM is a stock

Interval $\times$ calendar-spine joins, never SUM of events

2

Model the full lifecycle

A soft delete is still cost

3

Name metrics explicitly

Scope/basis drift can silently 5x a number in weeks

4

One owner, one query

Per metric, reconciled before it ships

***“The goal isn't one perfect storage number.
It's one trusted number everyone can stand
behind.”***

Questions?